SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: MLA03

COURSE CODE: Reinforcement learning

COURSE OUTCOME COVERED

CO1*: Analyze reinforcement learning frameworks and Markov Decision Process concepts to model decision-making problems. (BL2, BL3)*

Assessment Tool Weightage: 50%

Dr. G. A. Senthil

QUESTIONS: 10 **Total Marks: 50**

Annexure A

RL Basic Experiments demo with Keras and TensorFlow using Anaconda navigator

Duration: 2hrs

Code of Conduct:

I ***K.B.S Pradeep / 192325112*** *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*

I understand that any violation of academic integrity rules will result in disciplinary action.

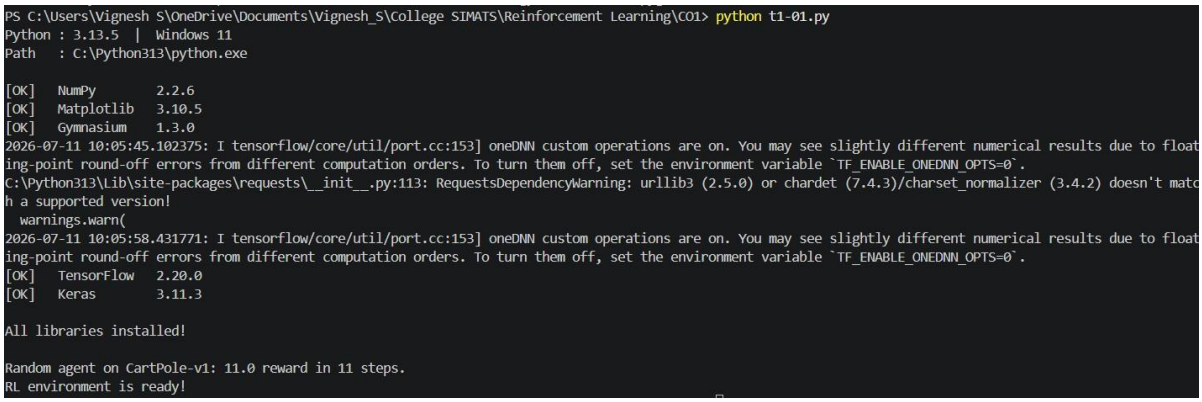
Signature of the student: **K.B.S Pradeep**

EXP 01:

```
import sys
import platform
import importlib

modules = {
    "NumPy": "numpy",
    "Matplotlib": "matplotlib",
    "Gymnasium": "gymnasium",
    "TensorFlow": "tensorflow",
    "Keras": "keras"
}

print("Python Version :", platform.python_version())
print("Operating System :", platform.system(), platform.release())
print("Python Path :", sys.executable)

missing = []

for name, module in modules.items():
    spec = importlib.util.find_spec(module)

    if spec is not None:
        lib = importlib.import_module(module)
        version = getattr(lib, "__version__", "Unknown")
        print(f"{name} -> Installed (Version: {version})")
    else:
        print(f"{name} -> Not Installed")
        missing.append(name)

if not missing:
    print("\nAll required libraries are available.")
```

```
    import gymnasium as gym

    env = gym.make("CartPole-v1")
    observation, info = env.reset(seed=42)

    total_reward = 0

    while True:
        action = env.action_space.sample()
        observation, reward, terminated, truncated, info = env.step(action)

        total_reward += reward

        if terminated or truncated:
            break

    env.close()

    print(f"CartPole test completed with reward: {total_reward}")
    print("Reinforcement Learning environment is working correctly.")
else:
  print("\nMissing libraries:", ", ".join(missing))
```

Output:



EXP 02:

```
import gymnasium as gym

environments = [
    ("FrozenLake-v1", {"is_slippery": False}),
    ("CartPole-v1", {}),
    ("MountainCar-v0", {})
]

for env_name, params in environments:
    print(f"\n--- {env_name} ---")

    env = gym.make(env_name, **params)

    print("Observation Space:", env.observation_space)
    print("Action Space:", env.action_space)

    state, _ = env.reset(seed=42)
    total_reward = 0

    for step in range(10):
        action = env.action_space.sample()
        state, reward, terminated, truncated, _ = env.step(action)

        total_reward += reward

        print(
            f"Step {step+1}: Action={action}, Reward={reward}, Done={terminated or
truncated}"
        )

        if terminated or truncated:
            break

    print("Total Reward:", total_reward)
    env.close()
```

OUTPUT:

```
===== FrozenLake-v1 =====
Observation space : Discrete(16)
Action space      : Discrete(4)
Initial state     : 0
Step  1 | action=2 | state=1 | reward=0 | done=False
Step  2 | action=2 | state=2 | reward=0 | done=False
Step  3 | action=0 | state=1 | reward=0 | done=False
Step  4 | action=3 | state=1 | reward=0 | done=False
Step  5 | action=3 | state=1 | reward=0 | done=False
Step  6 | action=3 | state=1 | reward=0 | done=False
Step  7 | action=0 | state=0 | reward=0 | done=False
Step  8 | action=1 | state=4 | reward=0 | done=False
Step  9 | action=2 | state=5 | reward=0 | done=True
Step 10 | action=1 | state=4 | reward=0 | done=False
Total reward: 0

===== CartPole-v1 =====
Observation space : Box([-4.8               -inf -0.41887903        -inf], [4.8               inf 0.41887903        inf], (4,), float32)
Action space      : Discrete(2)
Initial state     : [ 0.01369617 -0.02302133 -0.04590265 -0.04834723]
Step  1 | action=0 | state=[ 0.01323574 -0.21745604 -0.04686959  0.22950698] | reward=1.0 | done=False
Step  2 | action=1 | state=[ 0.00888662 -0.02169675 -0.04227945 -0.0775841 ] | reward=1.0 | done=False
Step  3 | action=1 | state=[ 0.00845269  0.174005   -0.04383114 -0.38330084] | reward=1.0 | done=False
Step  4 | action=0 | state=[ 0.01193279 -0.02046811 -0.05149715 -0.10475358] | reward=1.0 | done=False
Step  5 | action=1 | state=[ 0.01152342  0.17535256 -0.05359222 -0.41322866] | reward=1.0 | done=False
Step  6 | action=1 | state=[ 0.01503048  0.3711916  -0.0618568  -0.722314  ] | reward=1.0 | done=False
Step  7 | action=0 | state=[ 0.02245431  0.17697734 -0.07630308 -0.4497241 ] | reward=1.0 | done=False
Step  8 | action=0 | state=[ 0.02599385 -0.01698713 -0.08529756 -0.18203531] | reward=1.0 | done=False
Step  9 | action=0 | state=[ 0.02565411 -0.21079154 -0.08893827  0.08256733] | reward=1.0 | done=False
Step 10 | action=0 | state=[ 0.02143828 -0.40453345 -0.08728692  0.34591818] | reward=1.0 | done=False
Total reward: 10.0
```

EXP 03:

```
import numpy as np
import gymnasium as gym

env = gym.make("FrozenLake-v1", is_slippery=False)

states = env.observation_space.n
actions = env.action_space.n

Q = np.zeros((states, actions))

learning_rate = 0.8
discount = 0.95
epsilon = 1.0

scores = []

for ep in range(2000):

    state, _ = env.reset()
    total_reward = 0

    while True:

        # Choose action
        if np.random.rand() < epsilon:
            action = env.action_space.sample()
        else:
            action = np.argmax(Q[state])

        next_state, reward, terminated, truncated, _ = env.step(action)

        # Q-learning update
        best_next = np.max(Q[next_state])

        Q[state][action] = Q[state][action] + learning_rate * (
```

```
            reward + discount * best_next - Q[state][action]
        )

        state = next_state
        total_reward += reward

        if terminated or truncated:
            break

    scores.append(total_reward)

    # Reduce exploration
    epsilon *= 0.999
    if epsilon < 0.01:
        epsilon = 0.01

env.close()

print("Average reward (last 100 episodes):", round(np.mean(scores[-100:]), 2))

print("\nLearned Policy:")

arrows = ["<", "v", ">", "^"]

policy = np.argmax(Q, axis=1)

for i in range(0, 16, 4):
    print(" ".join(arrows[a] for a in policy[i:i+4]))
```

OUTPUT:

```
Average reward (last 100 episodes): 0.81

Learned policy (< v > ^):
  v  >  v  <
  v  <  v  <
  >  v  v  <
  <  >  >  <
```

EXP 04:

```
import numpy as np

np.random.seed(1)

# transitions[state][action] = list of (probability, next_state, reward)
transitions = {
  "Sleep": {"work": [(1.0, "Study", -1)], "rest": [(1.0, "Sleep", -2)]},
  "Study": {"work": [(0.8, "Pass", 10), (0.2, "Play", -1)], "rest": [(1.0, "Sleep", -1)]},
  "Play":  {"work": [(0.6, "Study", 0), (0.4, "Play", -1)], "rest": [(1.0, "Sleep", -2)]},
}
policy = {"Sleep": "work", "Study": "work", "Play": "work"}

def step(state, action):
  outcomes = transitions[state][action]
  i = np.random.choice(len(outcomes), p=[o[0] for o in outcomes])
  return outcomes[i][1], outcomes[i][2]

def run_episode(start="Sleep", max_steps=20):
  state, total = start, 0
  for _ in range(max_steps):
    action = policy[state]
    next_state, reward = step(state, action)
    total += reward
    print(f"  {state:<6} --({action})--> {next_state:<6} | reward = {reward}")
    state = next_state
    if state == "Pass":
      break
  print(f"  Total reward: {total}")
  return total

rewards = []
for ep in range(5):
  print(f"Episode {ep + 1}:")
  rewards.append(run_episode())
print(f"\nAverage reward over {len(rewards)} episodes: {np.mean(rewards):.2f}")
```

OUTPUT:

```
Episode 1:
  Sleep  --(work)--> Study  | reward = -1
  Study  --(work)--> Pass   | reward = 10
  Total reward: 9
Episode 2:
  Sleep  --(work)--> Study  | reward = -1
  Study  --(work)--> Pass   | reward = 10
  Total reward: 9
Episode 3:
  Sleep  --(work)--> Study  | reward = -1
  Study  --(work)--> Pass   | reward = 10
  Total reward: 9
Episode 4:
  Sleep  --(work)--> Study  | reward = -1
  Study  --(work)--> Pass   | reward = 10
  Total reward: 9
Episode 5:
  Sleep  --(work)--> Study  | reward = -1
  Study  --(work)--> Pass   | reward = 10
  Total reward: 9

Average reward over 5 episodes: 9.00
```

EXP 05:

```
import numpy as np

SIZE = 4
N = SIZE * SIZE
ACTIONS = [0, 1, 2, 3]  # Up, Down, Left, Right
TERMINAL = [0, N - 1]
REWARD = -1

def move(state, action):
    if state in TERMINAL:
     return state
  row, col = divmod(state, SIZE)
  if action == 0:
    row = max(row - 1, 0)
  elif action == 1:
    row = min(row + 1, SIZE - 1)
  elif action == 2:
    col = max(col - 1, 0)
  elif action == 3:
    col = min(col + 1, SIZE - 1)
  return row * SIZE + col

def policy_evaluation(gamma=1.0, theta=1e-4):
  V = np.zeros(N)
  iterations = 0
  while True:
    delta = 0
    for s in range(N):
       if s in TERMINAL:
```

```
            continue
        v = V[s]
        V[s] = np.mean([REWARD + gamma * V[move(s, a)] for a in ACTIONS])
        delta = max(delta, abs(v - V[s]))
      iterations += 1
      if delta < theta:
        break
  print(f"Policy Evaluation converged in {iterations} iterations.")
  return V

def  value_iteration(gamma=1.0,  theta=1e-4):
  V = np.zeros(N)
  iterations = 0
  while True:
    delta = 0
    for s in range(N):
      if s in TERMINAL:
        continue
      v = V[s]
      V[s] = max(REWARD + gamma * V[move(s, a)] for a in ACTIONS)
      delta = max(delta, abs(v - V[s]))
    iterations += 1
    if delta < theta:
      break
  print(f"Value Iteration converged in {iterations} iterations.")
  return V

V_pi = policy_evaluation()
print("V(s) for random policy:")
print(np.round(V_pi.reshape(SIZE, SIZE), 2), "\n")

V_star = value_iteration()
print("Optimal V*(s):")
print(np.round(V_star.reshape(SIZE, SIZE), 2), "\n")

symbols = {0: "U", 1: "D", 2: "L", 3: "R"}
print("Optimal policy:")
for s in range(N):
  a = int(np.argmax([REWARD + V_star[move(s, a)] for a in ACTIONS]))
  end = "\n" if s % SIZE == SIZE - 1 else "  "
  print("*" if s in TERMINAL else symbols[a], end=end)
```

OUTPUT:

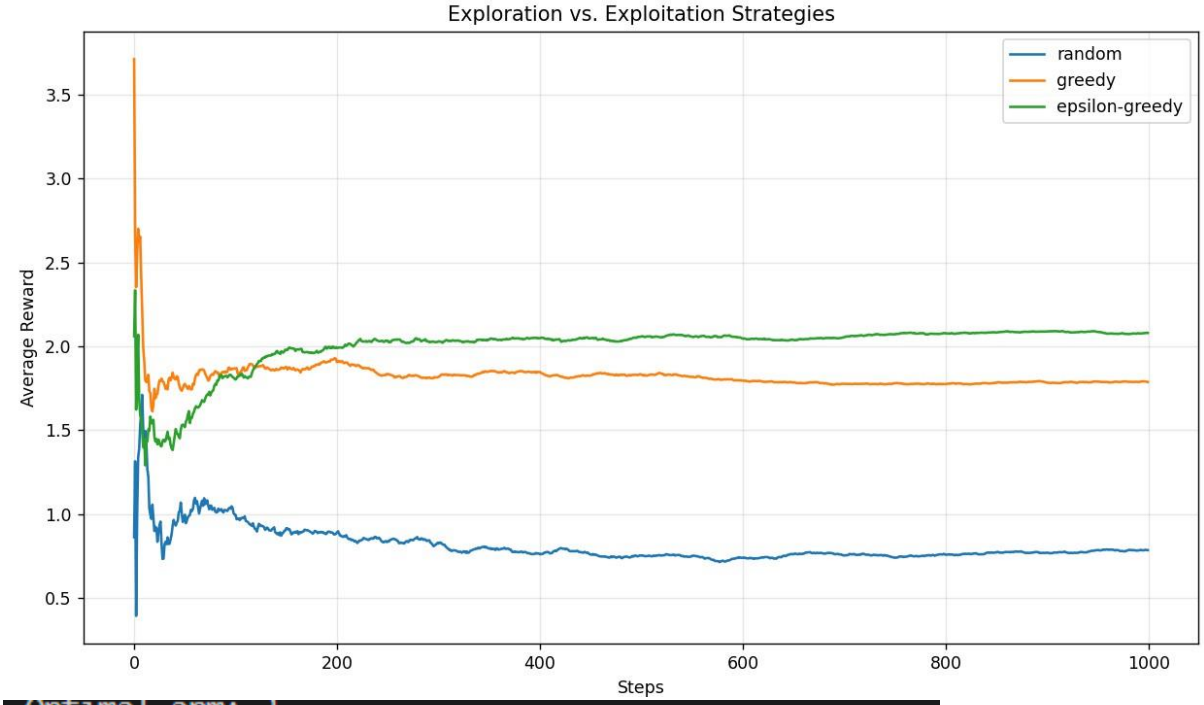
```
Policy Evaluation converged in 114 iterations.
V(s) for random policy:
[[  0. -14. -20. -22.]
 [-14. -18. -20. -20.]
 [-20. -20. -18. -14.]
 [-22. -20. -14.   0.]]

Value Iteration converged in 4 iterations.
Optimal V*(s):
[[ 0. -1. -2. -3.]
 [-1. -2. -3. -2.]
 [-2. -3. -2. -1.]
 [-3. -2. -1.  0.]]

Optimal policy:
*  L  L  D
U  U  U  D
U  U  D  D
U  R  R  *
```

EXP 06:

```
import numpy as np
import matplotlib.pyplot as plt

np.random.seed(0)
K, STEPS = 10, 1000
true_values = np.random.normal(0, 1, K)

def run(strategy, epsilon=0.1):
  q = np.zeros(K)
  counts = np.zeros(K)
  rewards = np.zeros(STEPS)
  for t in range(STEPS):
    if strategy == "random":
      a = np.random.randint(K)
    elif strategy == "greedy":
      a = int(np.argmax(q))
    else: # epsilon-greedy
      a = np.random.randint(K) if np.random.random() < epsilon else int(np.argmax(q))
    reward = np.random.normal(true_values[a], 1)
    counts[a] += 1
    q[a] += (reward - q[a]) / counts[a]
    rewards[t] = reward
  return rewards
```

```
print(f"Optimal arm: {np.argmax(true_values)}\n")
plt.figure(figsize=(10, 6))
for strat in ["random", "greedy", "epsilon-greedy"]:
    rewards = run(strat)
    print(f"{strat:<15} average reward = {np.mean(rewards):.3f}")
    plt.plot(np.cumsum(rewards) / (np.arange(STEPS) + 1), label=strat)

plt.xlabel("Steps")
plt.ylabel("Average Reward")
plt.title("Exploration vs. Exploitation Strategies")
plt.legend()
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.savefig("t1-06_exploration_vs_exploitation.png", dpi=120)
plt.show()
```

OUTPUT:

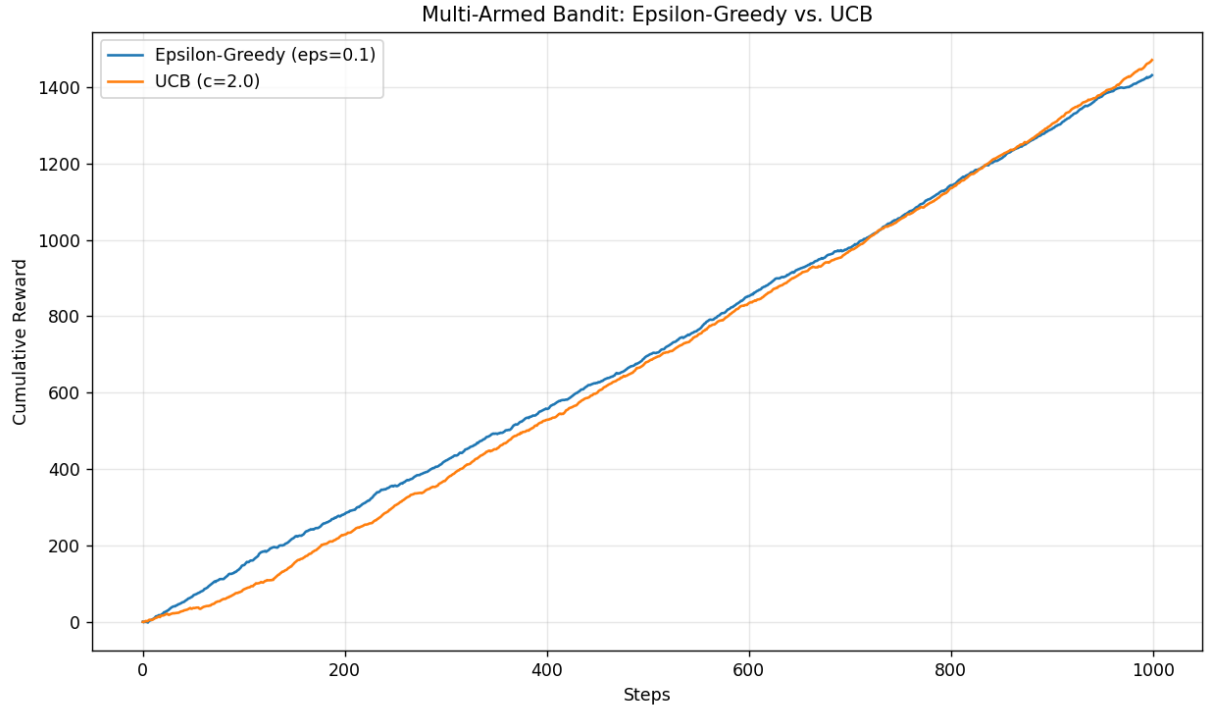



```
Optimal arm: 3

random          average reward = 0.785
greedy          average reward = 1.787
epsilon-greedy  average reward = 2.079
```

EXP 07:

```
import numpy as np
import matplotlib.pyplot as plt

np.random.seed(42)
K, STEPS = 10, 1000
true_values = np.random.normal(0, 1, K)

def epsilon_greedy(epsilon=0.1):
  q, counts, rewards = np.zeros(K), np.zeros(K), np.zeros(STEPS)
  for t in range(STEPS):
    a = np.random.randint(K) if np.random.random() < epsilon else int(np.argmax(q))
    reward = np.random.normal(true_values[a], 1)
    counts[a] += 1
    q[a] += (reward - q[a]) / counts[a]
    rewards[t] = reward
  return rewards

def ucb(c=2.0):
  q, counts, rewards = np.zeros(K), np.zeros(K), np.zeros(STEPS)
  for t in range(STEPS):
    if t < K:
      a = t
    else:
      a = int(np.argmax(q + c * np.sqrt(np.log(t + 1) / (counts + 1e-9))))
    reward = np.random.normal(true_values[a], 1)
    counts[a] += 1
    q[a] += (reward - q[a]) / counts[a]
    rewards[t] = reward
  return rewards

print(f"Optimal arm: {np.argmax(true_values)}\n")
eg, u = epsilon_greedy(), ucb()
print(f"Epsilon-Greedy | avg = {np.mean(eg):.3f} | total = {np.sum(eg):.1f}")
print(f"UCB          | avg = {np.mean(u):.3f} | total = {np.sum(u):.1f}")

plt.figure(figsize=(10, 6))
plt.plot(np.cumsum(eg), label="Epsilon-Greedy (eps=0.1)")
plt.plot(np.cumsum(u), label="UCB (c=2.0)")
plt.xlabel("Steps")
plt.ylabel("Cumulative Reward")
plt.title("Multi-Armed Bandit: Epsilon-Greedy vs. UCB")
plt.legend()
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.savefig("t1-07_bandit_comparison.png", dpi=120)
plt.show()
```

OUTPUT:



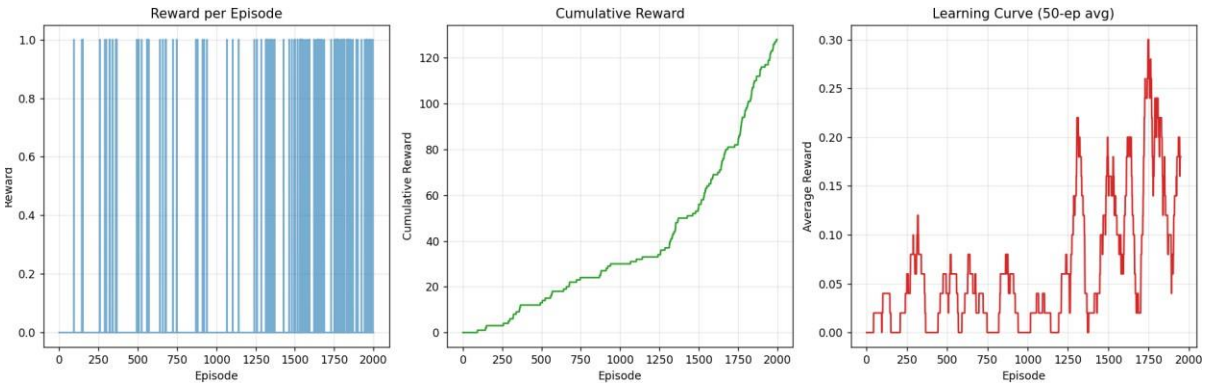

```
Optimal arm: 6

Epsilon-Greedy | avg = 1.431 | total = 1431.5
UCB            | avg = 1.471 | total = 1471.3
```

EXP 08:

```
import numpy as np
import matplotlib.pyplot as plt
import gymnasium as gym

np.random.seed(0)
env = gym.make("FrozenLake-v1", is_slippery=True)
q = np.zeros((env.observation_space.n, env.action_space.n))
alpha, gamma, epsilon = 0.8, 0.95, 1.0
rewards = []

for episode in range(2000):
  state, info = env.reset()
  done, total = False, 0
  while not done:
    action = env.action_space.sample() if np.random.random() < epsilon else
int(np.argmax(q[state]))
    next_state, reward, terminated, truncated, info = env.step(action)
    done = terminated or truncated
    q[state, action] += alpha * (reward + gamma * np.max(q[next_state]) - q[state,
action])
    state = next_state
```

```
        total += reward
    rewards.append(total)
    epsilon = max(0.01, epsilon * 0.999)

env.close()
print(f"Average reward (last 100): {np.mean(rewards[-100:]):.3f}")

smoothed = np.convolve(rewards, np.ones(50) / 50, mode="valid")
fig, axes = plt.subplots(1, 3, figsize=(16, 5))
axes[0].plot(rewards, color="tab:blue", alpha=0.6)
axes[0].set(title="Reward per Episode", xlabel="Episode", ylabel="Reward")
axes[1].plot(np.cumsum(rewards), color="tab:green")
axes[1].set(title="Cumulative Reward", xlabel="Episode", ylabel="Cumulative Reward")
axes[2].plot(smoothed, color="tab:red")
axes[2].set(title="Learning Curve (50-ep avg)", xlabel="Episode", ylabel="Average
Reward")
for ax in axes:
    ax.grid(True, alpha=0.3)
plt.tight_layout()
plt.savefig("t1-08_reward_visualization.png", dpi=120)
plt.show()
```

OUTPUT:



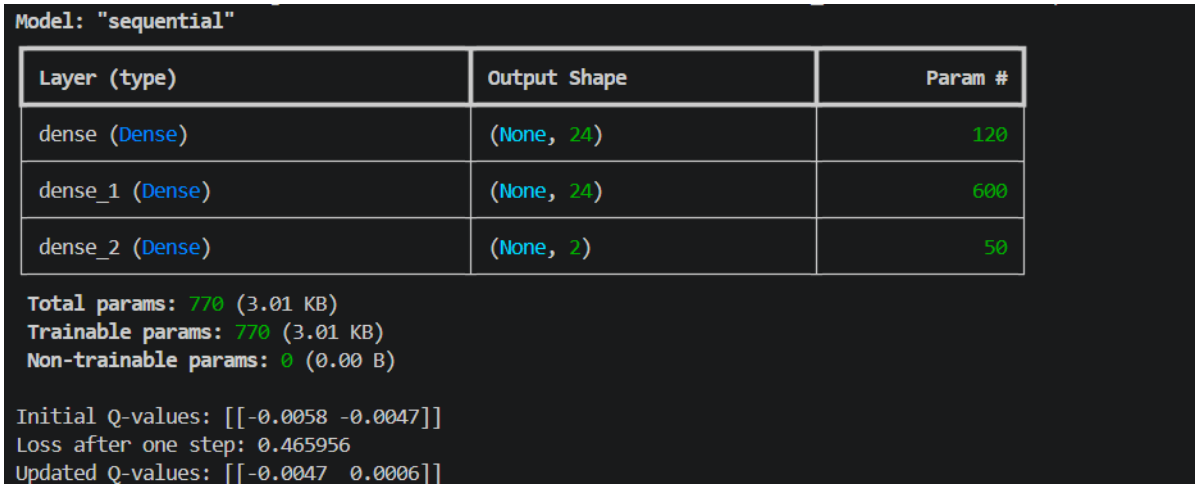

```
Average reward (last 100): 0.120
```

EXP 0G:

```
import numpy as np
import gymnasium as gym
from tensorflow import keras
from tensorflow.keras import layers

env = gym.make("CartPole-v1")
```

```
state_size = env.observation_space.shape[0]
action_size = env.action_space.n

model = keras.Sequential([
  keras.Input(shape=(state_size,)),
  layers.Dense(24, activation="relu"),
  layers.Dense(24, activation="relu"),
  layers.Dense(action_size, activation="linear"),
])
model.compile(optimizer=keras.optimizers.Adam(0.001), loss="mse")
model.summary()

state, info = env.reset(seed=0)
action = env.action_space.sample()
next_state, reward, terminated, truncated, info = env.step(action)
state = state.reshape(1, -1)
next_state = next_state.reshape(1, -1)

q_values = model.predict(state, verbose=0)
print("\nInitial Q-values:", np.round(q_values, 4))

gamma = 0.95
target = q_values.copy()
target[0][action] = reward + gamma * np.max(model.predict(next_state, verbose=0))

loss = model.fit(state, target, epochs=1, verbose=0).history["loss"][0]
print(f"Loss after one step: {loss:.6f}")
print("Updated Q-values:", np.round(model.predict(state, verbose=0), 4))
env.close()
```

OUTPUT:



Model: "sequential"

Layer (type)	Output Shape	Param #
dense (Dense)	(None, 24)	120
dense_1 (Dense)	(None, 24)	600
dense_2 (Dense)	(None, 2)	50

Total params: 770 (3.01 KB)
Trainable params: 770 (3.01 KB)
Non-trainable params: 0 (0.00 B)

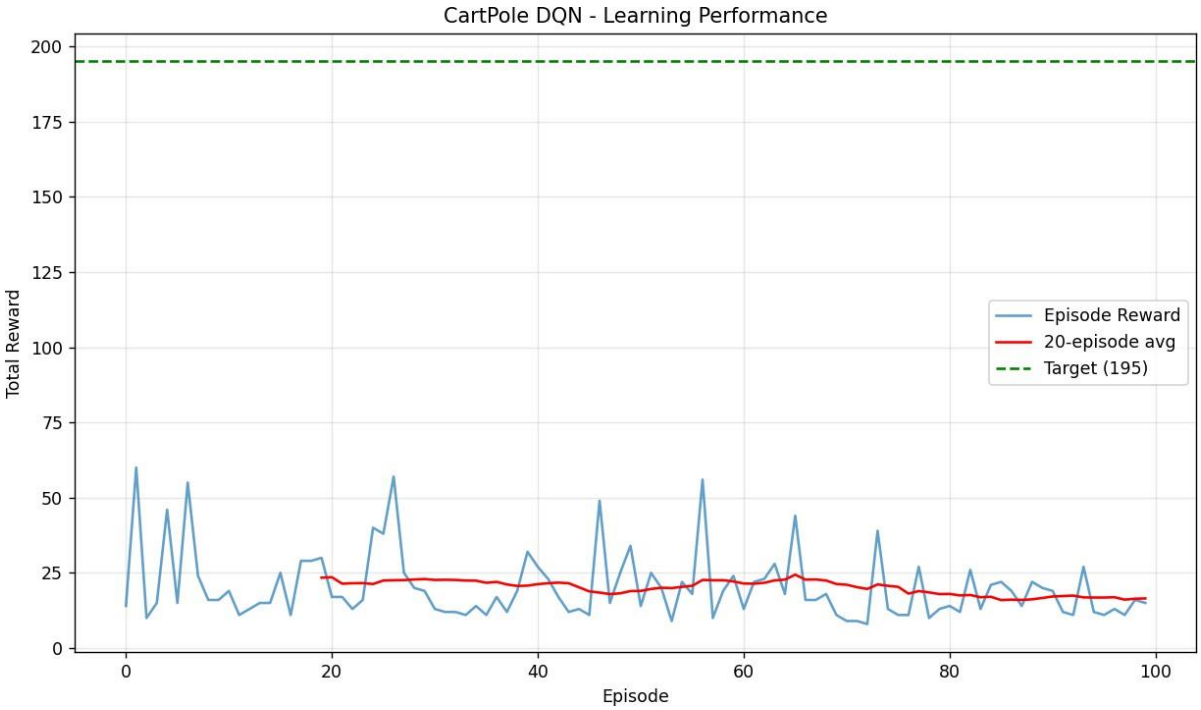
Initial Q-values: [[-0.0058 -0.0047]]
Loss after one step: 0.465956
Updated Q-values: [[-0.0047 0.0006]]

EXP 10:

```
import random
from collections import deque

import numpy as np
import matplotlib.pyplot as plt
import gymnasium as gym
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers

np.random.seed(0)
random.seed(0)
tf.random.set_seed(0)

class DQNAgent:
  def __init__(self, state_size, action_size):
    self.action_size = action_size
    self.memory = deque(maxlen=2000)
    self.gamma = 0.95
    self.epsilon = 1.0
    self.model = keras.Sequential([
      keras.Input(shape=(state_size,)),
      layers.Dense(24, activation="relu"),
      layers.Dense(24, activation="relu"),
      layers.Dense(action_size, activation="linear"),
    ])
    self.model.compile(optimizer=keras.optimizers.Adam(0.001),  loss="mse")

  def act(self, state):
    if np.random.rand() <= self.epsilon:
      return random.randrange(self.action_size)
    return int(np.argmax(self.model.predict(state, verbose=0)[0]))

  def replay(self, batch_size=32):
    if len(self.memory) < batch_size:
      return
    batch = random.sample(self.memory, batch_size)
    states = np.vstack([b[0] for b in batch])
    next_states = np.vstack([b[3] for b in batch])
    q = self.model.predict(states, verbose=0)
    q_next = self.model.predict(next_states, verbose=0)
    for i, (_, action, reward, _, done) in enumerate(batch):
      q[i][action] = reward if done else reward + self.gamma * np.max(q_next[i])
    self.model.fit(states, q, epochs=1, verbose=0)
    self.epsilon = max(0.01, self.epsilon * 0.995)
```

```
env = gym.make("CartPole-v1")
state_size = env.observation_space.shape[0]
agent = DQNAgent(state_size, env.action_space.n)
scores = []

for episode in range(100):
  state, info = env.reset()
  state = state.reshape(1, -1)
  done, total = False, 0
  while not done:
    action = agent.act(state)
    next_state, reward, terminated, truncated, info = env.step(action)
    done = terminated or truncated
    next_state = next_state.reshape(1, -1)
    agent.memory.append((state, action, reward, next_state, done))
    state = next_state
    total += reward
  agent.replay()
  scores.append(total)
  avg = np.mean(scores[-20:])
  print(f"Episode {episode + 1:>3} | reward = {total:>5.0f} | avg(20) = {avg:6.1f} | eps =
{agent.epsilon:.3f}")
  if avg >= 195 and len(scores) >= 20:
    print(f"\nSolved! Average {avg:.1f} >= 195")
    break

env.close()
print(f"\nBest reward: {max(scores):.0f} | Success rate (>=195):
{np.mean(np.array(scores) >= 195) * 100:.1f}%")

plt.figure(figsize=(10, 6))
plt.plot(scores, label="Episode Reward", alpha=0.7)
if len(scores) >= 20:
  moving = np.convolve(scores, np.ones(20) / 20, mode="valid")
  plt.plot(range(19, len(scores)), moving, color="red", label="20-episode avg")
plt.axhline(y=195, color="green", linestyle="--", label="Target (195)")
plt.xlabel("Episode")
plt.ylabel("Total Reward")
plt.title("CartPole DQN - Learning Performance")
plt.legend()
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.savefig("t1-10_cartpole_dqn.png", dpi=120)
plt.show()
```

OUTPUT:



Episode 1 | reward = 14 | avg(20) = 14.0 | eps = 1.000
Episode 2 | reward = 60 | avg(20) = 37.0 | eps = 0.995
Episode 3 | reward = 10 | avg(20) = 28.0 | eps = 0.990
Episode 4 | reward = 15 | avg(20) = 24.8 | eps = 0.985
Episode 5 | reward = 46 | avg(20) = 29.0 | eps = 0.980
Episode 6 | reward = 15 | avg(20) = 26.7 | eps = 0.975
Episode 7 | reward = 55 | avg(20) = 30.7 | eps = 0.970
Episode 8 | reward = 24 | avg(20) = 29.9 | eps = 0.966
Episode 9 | reward = 16 | avg(20) = 28.3 | eps = 0.961
Episode 10 | reward = 16 | avg(20) = 27.1 | eps = 0.956
Episode 11 | reward = 19 | avg(20) = 26.4 | eps = 0.951
Episode 12 | reward = 11 | avg(20) = 25.1 | eps = 0.946
Episode 13 | reward = 13 | avg(20) = 24.2 | eps = 0.942
Episode 14 | reward = 15 | avg(20) = 23.5 | eps = 0.937
Episode 15 | reward = 15 | avg(20) = 22.9 | eps = 0.932
Episode 16 | reward = 25 | avg(20) = 23.1 | eps = 0.928
Episode 17 | reward = 11 | avg(20) = 22.4 | eps = 0.923
Episode 18 | reward = 29 | avg(20) = 22.7 | eps = 0.918
Episode 19 | reward = 29 | avg(20) = 23.1 | eps = 0.914
Episode 20 | reward = 30 | avg(20) = 23.4 | eps = 0.909
Episode 21 | reward = 17 | avg(20) = 23.6 | eps = 0.905
Episode 22 | reward = 17 | avg(20) = 21.4 | eps = 0.900
Episode 23 | reward = 13 | avg(20) = 21.6 | eps = 0.896
Episode 24 | reward = 16 | avg(20) = 21.6 | eps = 0.891
Episode 25 | reward = 40 | avg(20) = 21.3 | eps = 0.887
Episode 26 | reward = 38 | avg(20) = 22.4 | eps = 0.882

```
Episode 27 | reward =  57 | avg(20) =  22.6 | eps = 0.878
Episode 28 | reward =  25 | avg(20) =  22.6 | eps = 0.873
Episode 29 | reward =  20 | avg(20) =  22.8 | eps = 0.869
Episode 30 | reward =  19 | avg(20) =  22.9 | eps = 0.865
Episode 31 | reward =  13 | avg(20) =  22.6 | eps = 0.860
Episode 32 | reward =  12 | avg(20) =  22.7 | eps = 0.856
Episode 33 | reward =  12 | avg(20) =  22.6 | eps = 0.852
Episode 34 | reward =  11 | avg(20) =  22.4 | eps = 0.848
Episode 35 | reward =  14 | avg(20) =  22.4 | eps = 0.843
Episode 36 | reward =  11 | avg(20) =  21.7 | eps = 0.839
Episode 37 | reward =  17 | avg(20) =  22.0 | eps = 0.835
Episode 38 | reward =  12 | avg(20) =  21.1 | eps = 0.831
Episode 39 | reward =  19 | avg(20) =  20.6 | eps = 0.827
Episode 40 | reward =  32 | avg(20) =  20.8 | eps = 0.822
Episode 41 | reward =  27 | avg(20) =  21.2 | eps = 0.818
Episode 42 | reward =  23 | avg(20) =  21.6 | eps = 0.814
Episode 43 | reward =  17 | avg(20) =  21.8 | eps = 0.810
Episode 44 | reward =  12 | avg(20) =  21.6 | eps = 0.806
Episode 45 | reward =  13 | avg(20) =  20.2 | eps = 0.802
Episode 46 | reward =  11 | avg(20) =  18.9 | eps = 0.798
Episode 47 | reward =  49 | avg(20) =  18.4 | eps = 0.794
Episode 48 | reward =  15 | avg(20) =  17.9 | eps = 0.790
Episode 49 | reward =  25 | avg(20) =  18.2 | eps = 0.786
Episode 50 | reward =  34 | avg(20) =  18.9 | eps = 0.782
Episode 51 | reward =  14 | avg(20) =  19.0 | eps = 0.778
Episode 52 | reward =  25 | avg(20) =  19.6 | eps = 0.774
Episode 53 | reward =  20 | avg(20) =  20.1 | eps = 0.771
Episode 54 | reward =   9 | avg(20) =  19.9 | eps = 0.767
Episode 55 | reward =  22 | avg(20) =  20.4 | eps = 0.763
Episode 56 | reward =  18 | avg(20) =  20.7 | eps = 0.759
Episode 57 | reward =  56 | avg(20) =  22.6 | eps = 0.755
Episode 58 | reward =  10 | avg(20) =  22.6 | eps = 0.751
Episode 59 | reward =  19 | avg(20) =  22.6 | eps = 0.748
Episode 60 | reward =  24 | avg(20) =  22.1 | eps = 0.744
Episode 61 | reward =  13 | avg(20) =  21.4 | eps = 0.740
Episode 62 | reward =  22 | avg(20) =  21.4 | eps = 0.737
Episode 63 | reward =  23 | avg(20) =  21.7 | eps = 0.733
Episode 64 | reward =  28 | avg(20) =  22.5 | eps = 0.729
Episode 65 | reward =  18 | avg(20) =  22.8 | eps = 0.726
Episode 66 | reward =  44 | avg(20) =  24.4 | eps = 0.722
Episode 67 | reward =  16 | avg(20) =  22.8 | eps = 0.718
Episode 68 | reward =  16 | avg(20) =  22.8 | eps = 0.715
Episode 69 | reward =  18 | avg(20) =  22.4 | eps = 0.711
Episode 70 | reward =  11 | avg(20) =  21.3 | eps = 0.708
Episode 71 | reward =   9 | avg(20) =  21.1 | eps = 0.704
Episode 72 | reward =   9 | avg(20) =  20.2 | eps = 0.701
Episode 73 | reward =   8 | avg(20) =  19.6 | eps = 0.697
```

```
Episode  74 | reward =   39 | avg(20) =  21.1 | eps = 0.694
Episode  75 | reward =   13 | avg(20) =  20.7 | eps = 0.690
Episode  76 | reward =   11 | avg(20) =  20.4 | eps = 0.687
Episode  77 | reward =   11 | avg(20) =  18.1 | eps = 0.683
Episode  78 | reward =   27 | avg(20) =  18.9 | eps = 0.680
Episode  79 | reward =   10 | avg(20) =  18.5 | eps = 0.676
Episode  80 | reward =   13 | avg(20) =  17.9 | eps = 0.673
Episode  81 | reward =   14 | avg(20) =  18.0 | eps = 0.670
Episode  82 | reward =   12 | avg(20) =  17.5 | eps = 0.666
Episode  83 | reward =   26 | avg(20) =  17.6 | eps = 0.663
Episode  84 | reward =   13 | avg(20) =  16.9 | eps = 0.660
Episode  85 | reward =   21 | avg(20) =  17.1 | eps = 0.656
Episode  86 | reward =   22 | avg(20) =  15.9 | eps = 0.653
Episode  87 | reward =   19 | avg(20) =  16.1 | eps = 0.650
Episode  88 | reward =   14 | avg(20) =  16.0 | eps = 0.647
Episode  89 | reward =   22 | avg(20) =  16.2 | eps = 0.643
Episode  90 | reward =   20 | avg(20) =  16.6 | eps = 0.640
Episode  91 | reward =   19 | avg(20) =  17.1 | eps = 0.637
Episode  92 | reward =   12 | avg(20) =  17.3 | eps = 0.634
Episode  93 | reward =   11 | avg(20) =  17.4 | eps = 0.631
Episode  94 | reward =   27 | avg(20) =  16.9 | eps = 0.627
Episode  95 | reward =   12 | avg(20) =  16.8 | eps = 0.624
Episode  96 | reward =   11 | avg(20) =  16.8 | eps = 0.621
Episode  97 | reward =   13 | avg(20) =  16.9 | eps = 0.618
Episode  98 | reward =   11 | avg(20) =  16.1 | eps = 0.615
Episode  99 | reward =   16 | avg(20) =  16.4 | eps = 0.612
Episode 100 | reward =   15 | avg(20) =  16.5 | eps = 0.609

Best reward: 60 | Success rate (>=195): 0.0%
```

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25%	Demonstrates thorough, accurate understanding of concepts	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor understanding; major misconceptions
Explanation	25%	Detailed, well-explained answers with relevant examples	Clear explanation with adequate details	Limited explanation; lacks depth	Poor or unclear explanation
Application	20%	Effectively applies concepts with strong analytical ability	Applies concepts with minor errors	Limited application with weak analysis	No application or incorrect analysis
Organization	15%	Well-structured answers with logical flow and coherence	Organized with minor issues in flow	Basic structure, lacks clarity	Poor organization, incoherent answers
Accuracy	10%	Completely accurate and covers all required points	Mostly accurate with minor omissions	Partially correct with missing points	Incorrect answers with major gaps
Clarity	5%	Clear, neat, professional presentation	Understandable with minor issues	Limited clarity, readability issues	Poorly written, difficult to understand